

Set up your own Hope Beacon

Four steps. No code to write.

This is the fast path from a clone to a working church app that real people sign into from their own phones. The whole backend already ships in this repository — the schema, the security rules, the sign-in gateway. Nothing here asks you to build one.

If you want to understand *how* a backend for this app is designed, or to build one on a different stack, read [BUILD-YOUR-OWN.md](#). That document is a course. This one is the instructions.

The app has a page that does all of this with you and checks your work as you go. Start the app and open `/setup` — it tells you which step you are on rather than making you guess.

Before you start

You need [Node 22 or newer](https://nodejs.org) and a free account with a Postgres host that also provides authentication. The migrations in

covers what changes on other hosts.

Total time: about twenty minutes, most of it waiting for a database to start.

Step 1 — Create a database and connect it

Create a new project with your host. When it finishes starting, open its **Settings** → **API** page and leave it open — the next command needs two values from it.

Then, in this folder:

```
npm run setup
```

It asks two questions, checks your answers, and writes the settings file for you. There is no file to create by hand and no variable name to spell.

It will stop you if you paste the wrong key. That settings page shows two: one is meant to be published to every browser, the other bypasses every security rule in your database. They look alike and sit next to each other. The setup command reads which one you pasted and refuses the dangerous one.

Step 2 — Create the tables

Open your project's SQL editor and run every file in `supabase/migrations/`, **in filename order**. Each one builds on the last, so the order is not optional.

File	What it adds
<code>0001_core_schema.sql</code>	People, churches, pairings, messages, lessons — and the security rules
<code>0002_invitations.sql</code>	How somebody is invited and joins
<code>0003_invite_approval_gate.sql</code>	Nobody gets in until a Director approves them
<code>0004_live_api_permissions.sql</code>	What a signed-in browser is allowed to call
<code>0005_platform_function_acl.sql</code>	Locks the platform functions to the roles that need them
<code>20260816130240_approval_revocation_gate.sql</code>	Taking approval away actually removes access

Read `0001` before you run it. Everything protecting your congregation's privacy is in that file: a Guide sees their own people and nobody else's, an Explorer sees only themselves, a Director sees their own church. It is enforced by the database, not by the screens, which is why it holds even if somebody calls the API directly.

Step 3 — Restart the app

```
npm run dev
```

Settings are read once at startup, so a running app will not notice the file you just wrote.

The front door changes. Instead of a list of sample people to explore as, you get a real e-mail and password sign-in — the same screen your church members will use.

Step 4 — Make yourself the first administrator

There is no public sign-up. The app is invitation-only by design, which leaves the very first account a chicken-and-egg problem: nobody exists to invite you. Create it directly, once:

1. In your project, open **Authentication** and add a user with your e-mail address and a password.

1. In the SQL editor, find that person in `profiles` and set their role to `admin` and their approval flag to true.

Now sign in. From here everybody else joins by invitation, which is how it is meant to work — you should never need to touch the database to add a person again.

Before real people go in

Do not skip this. The rules are the product.

Try to break in as your least-privileged user. Sign in as an Explorer and attempt to reach what they should not: another person's conversation, the church list, the Director screens. You should be refused every time, by the database rather than by a missing button.

Or prove it in SQL, which is faster and more thorough:

```
# run against your database
docs/examples/prove-the-rules.sql
```

Check `/setup` one last time. It reports what it can actually see, so a green result there means the app really is talking to your database — not that it thinks it should be.

When something is wrong

Open `/setup` first. It names the specific step rather than reporting a general failure.

What you see	What it usually means
Still showing sample people	The app has not been restarted since step 1, or the settings file was not written
"Cannot reach your database"	The address is wrong, the project is paused, or the browser blocked the request
"Connected, but the tables are missing"	Step 2 has not been run, or it stopped partway
Sign-in says the account is not ready	The account exists but has not been approved — step 4, second half

Taking it further

- [EMAIL.md](#) — sending invitations by mail, with any provider (optional)
- [BACKENDS.md](#) — using a host other than the one these migrations assume
- [SECURITY.md](#) — what the rules guarantee, and what they do not
- [BUILD-YOUR-OWN.md](#) — the long course, if you want to build the backend yourself
- [CONTRIBUTING.md](#) — how to work on the project itself